

Issue #47 Readiness

08/09/2026 14:59:10

Sport-Analytics-Tool-main.zip is all the work done here for me to be able to do issue #47?

Thinking 🌐 Inspecting Repository Contents for Issue #47 ● Inspected repository structure, dependencies,...
so what should i start with? 58?

Yes - start with #58 first. That is the cleanest prerequisite for your eventual #47 work. I'd do your work in this order...

Sport-Analytics-Tool (5).zip ok, so im taking #58, here is the schema of our current db, does that help? i need step...

Thinking 🌐 Inspecting newest ZIP and Git issue #58 ● Inspected repository ZIP, Git issue, contracts, documentation...
confirming none of the steps u did used supabases apis? remember proj says we (including AI) need to hand write...

Thinking ● Confirmed no Supabase-generated APIs were used ✓ Worked for 10s Yes - none of the #58 steps I gave...
Update `docs/api/overview.md`. Right now it says: Pagination: cursor pagination for large or changing collection...

Thinking 🌐 Inspecting API Overview Documentation ● Inspected and read API overview documentation ✓ Worked...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> python -m mkdocs build --strict C:\Python314\python.exe...

Thinking 🌐 Inspecting project dependencies for MkDocs setup ● Inspected MkDocs dependencies and requirements...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check > sport-analytics-tool\@0.1.0 check...

Thinking 🌐 Inspecting repository formatting issues and next steps ● Validating targeted files I'm noting that...

S C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run test > sport-analytics-tool\@0.1.0 test >...

Thinking 🌐 Inspecting Package Dependencies and Lockfiles ● Fixing the dependency I'm treating this as a stale...

ok, so now im ready for updating the ai usage stuff, and then commit? please give me step by step

Thinking 🌐 Inspecting latest repository AI conventions ● Updating project records I'm identifying Shayna's project...



Sport-Analytics-Tool-main.zip

is all the work done here for me to be able to do issue #47?

8/9/2026, 2:16:41 PM



Thinking

Inspecting Repository Contents for Issue #47

- Inspected repository structure, dependencies, migrations, and readiness for Issue #47

Remembering

- Assessing dependencies

I'm checking whether the public read APIs can start safely. The main blockers appear to be the unfinished database schema, repository layer, shared contracts, OpenAPI baseline, and seed data; otherwise I'd risk building against temporary structures.

- Assessed dependencies

- Clarifying readiness

I'm separating "can begin implementation" from "is ready to complete safely." The backend scaffold exists, but I still need the schema, repository layer, shared contracts, OpenAPI baseline, and seed data before finalizing the public read APIs.

- Clarified readiness

- Identified COMS3011A issue #47 and reviewed its criteria and dependencies

✓ Worked for 1m 44s

Not quite. I reviewed the current uploaded `Sport-Analytics-Tool-main.zip` against your Sprint 1 dependency plan. **You are close, but I would not start implementing #47 as a merge-ready issue yet.**

Issue #47 = "Implement public competition, fixture and participant read APIs." This is important because the project brief specifically requires a **hand-written API**, and the Sport Analytics brief makes the API a core product through which consumers read fixture/event/statistical data. `sport_analytics.pdf` `project_briefs.pdf`

What is already ready ✓

Your repo has a good base for #47:

- Express backend exists.
- `/api/v1` routing convention exists.
- `GET /api/v1/health` proves the API scaffold works.
- Supabase PostgreSQL/migration tooling exists.
- The current event migration already contains useful entities:
 - `competition`
 - `fixture`
 - `team`
 - `person`
 - `fixture_team`
 - `fixture_squad`
- There is already an index for fixture discovery:



```
fixture_competition_season_idx
```

- Your architecture explicitly says the backend is the only application-data boundary.
- Public-read endpoints **do not require authentication**, so you do **not** need to wait for all the submitter/authentication work.

So you **do not need #44, #45, #49, #50 or #51 completed before #47**.

What is still missing ⚠️

There are four things I would want resolved first:

Issue	Current repo state	#47 impact
#46 - domain DB schema	Partly present , mostly through the #27 event migration, but the planned competition/season/competitor/participant schema has not clearly been completed as #46	🔴 Important
#58 - API contracts	Not present. <code>packages/contracts</code> currently only contains the health contract	🔴 Important
#62 - backend repository/transaction foundation	Not implemented in <code>apps/backend/src</code> ; there is currently no production DB pool/repository layer	🔴 Biggest blocker
#63 - representative seed/reference dataset	<code>database/seeds/</code> only contains a README	🟡 Needed for proper testing/demo

And **#59 - OpenAPI baseline** also appears unfinished: I found no OpenAPI/Swagger specification in the repository. I would ideally get that in place before or alongside #47.

The strongest evidence is your own dependency order:

```
Priority A
#58 #59 ...

↓

Priority B
#46 #62 #63

↓

Priority E
#47
```

That order exists specifically so #47 doesn't have to invent its database access and API shapes itself.

The main problem is #62

Right now `apps/backend/src` essentially contains:

```
auth/
config/
middleware/
routes/
  auth.routes.ts
  health.routes.ts
```

There is **no actual database repository layer yet**.

Your own `apps/backend/src/modules/README.md` says modules should normally contain:

```
route
controller
service
repository
validation schemas
tests
```

and specifically says:

database access belongs behind repositories.

If you started #47 now, you would either have to:

1. implement the repository foundation as part of #47, which steals scope from #62, or
2. temporarily query PostgreSQL directly inside controllers/routes, which goes against the architecture you've documented.

Neither is ideal.

So what should happen?

I would use this order:

```
#58 API contracts
↓
#46 finalise public domain schema
↓
#62 DB connection/repository foundation
↓
#63 seed/reference data
↓
#47 PUBLIC READ API ← YOU
↓
#48 public frontend
```

#59 OpenAPI can happen before #47 or in parallel, provided the contract is agreed before your endpoints are finalised.

Can you start anything on #47 now?

Yes - you can prepare it, but I wouldn't code the final implementation yet.

Once those prerequisites land, your #47 work should be fairly clean. It will likely become modules along the lines of:

```
apps/backend/src/modules/
├─ competitions/
│   ├─ competition.routes.ts
│   ├─ competition.controller.ts
│   ├─ competition.service.ts
│   └─ competition.repository.ts
│
├─ fixtures/
│   ├─ fixture.routes.ts
│   ├─ fixture.controller.ts
│   ├─ fixture.service.ts
│   └─ fixture.repository.ts
│
└─ participants/
    ├─ participant.routes.ts
    └─ participant.controller.ts
```

```
└─ participant.service.ts
└─ participant.repository.ts
```

with endpoints approximately like:

```
GET /api/v1/competitions

GET /api/v1/competitions/:competitionId

GET /api/v1/fixtures

GET /api/v1/fixtures/:fixtureId

GET /api/v1/fixtures/:fixtureId/participants
```

and **no bearer token required**.

Then #67 deals with the ordered delivery/event endpoints, #52 deals with statistics, and #48 consumes your endpoints from the frontend.

Bottom line

No - I would wait before properly doing #47.

You are not waiting for the whole team or the entire vertical slice. The specific things I would wait for are:

#58 → #46 → #62, with **#63 highly desirable** and **#59 either before or alongside your work**.

The biggest one is **#62**. Once the repository/database-access foundation is merged, #47 becomes a sensible standalone issue rather than you having to build somebody else's prerequisite inside your PR.

If you want, I can also go through the uploaded repo and tell you **exactly which currently open issue should be completed next before you personally start #47**, so you know who you're waiting on and whether you can begin today.

8/9/2026, 2:16:43 PM



so what should i start with? 58?

8/9/2026, 2:20:37 PM



Yes - start with #58 first. That is the cleanest prerequisite for your eventual **#47** work.

I'd do your work in this order:

#58 → #47

while the other team members complete **#46 / #62 / #63** in parallel.

Why #58 is the best one for you

#58 establishes the **API contracts** - i.e. what data the backend will return and the TypeScript/Zod shapes shared between the frontend and backend.

That is exactly what you need before implementing public endpoints in #47. It also fits the project requirement that your API be deliberately designed and hand-written rather than emerging accidentally from your database structure. `project_briefs.pdf`

The Sport Analytics brief says consumers need to be able to read **fixtures, events and derived statistics**, filter the data, page through larger results, and rely on stable identifiers. `sport_analytics.pdf` So defining those public-facing shapes first is good architecture.

Your path should therefore look like

```
YOU
#58 Define shared API contracts
      ↓
wait for / coordinate with
#46 Domain database schema
#62 Repository / DB access foundation
#63 Seed data
      ↓
YOU
#47 Implement public competition,
    fixture and participant APIs
```

You **don't need to sit idle waiting for #46/#62**. Do #58 now.

For #58, I would expect you to work mainly inside:

```
packages/contracts/
```

and define things roughly like:

```
Competition
Fixture
Participant

CompetitionResponse
CompetitionListResponse

FixtureResponse
FixtureListResponse

ParticipantResponse
ParticipantListResponse
```

Pagination
API error response

Potentially with Zod schemas + inferred TypeScript types, depending on the convention already established by your `health` contract.

One important thing

Don't start inventing the exact fields blindly.

Your contracts should line up with the schema being designed in **#46**. So while you can own #58, speak to whoever owns #46 and agree on things such as:

```
Competition
- id
- name
- season
- sport?

Fixture
- id
- competitionId
- scheduledAt
- status
- venue?
- participants/teams?

Participant
- id
- name
- type?
```

The precise fields should come from the team's agreed cricket/domain model, rather than you locking everyone into something independently.

What I'd do right now

Assign yourself #58 and create a branch for it.

Something like:

```
git checkout main
git pull
git checkout -b feat/58-api-contracts
```

Then we can do **#58 properly together from the issue description onward** - including the exact files to create, code, tests, AI declaration/update, commit message and PR description.

So yes: **#58 is your next move.**

8/9/2026, 2:20:42 PM



Sport-Analytics-Tool (5).zip

ok, so im taking #58, here is the schema of our current db, does that help?

i need step by step of what i need to do for 58!

u sure its not in the github? ;et me give u the newest git

Table `pgmigrations`

Columns

Name	Type	Constraints
-----	-----	-----
`id`	`int4`	Primary
`name`	`varchar`	
`run_on`	`timestamp`	

Table `database\_health`

Baseline table proving the migration mechanism works, and providing a target for the scheduled keep-alive query. Holds no product data.

Columns

Name	Type	Constraints
-----	-----	-----
`id`	`int4`	Primary
`initialised_at`	`timestamptz`	
`note`	`text`	

Table `dismissal\_kind`

Controlled vocabulary for dismissals. A lookup table rather than an enumeration because the set is open: two kinds present in the full corpus were absent from the earlier subset, and extending an enumeration requires a migration.

Columns

Name	Type	Constraints
-----	-----	-----
`code`	`text`	Primary
`display_name`	`text`	
`credits_bowler`	`bool`	
`is_retirement`	`bool`	

Table `person`

Columns

Name	Type	Constraints
-----	-----	-----
`person_id`	`int8`	Primary Identity
`source_ref`	`text`	Unique
`display_name`	`text`	

Table `person\_alias`



Every name ever observed for a person, so that a name in an older submission still resolves after the source renames or disambiguates it.

Columns

Name	Type	Constraints
-----	-----	-----
`person_id`	`int8`	Primary
`name`	`text`	Primary
`first_seen`	`date`	Nullable

Table `official`

Officials are kept separate from person because the source provides no stable identifier for them. Merging them on name would assert an identity that cannot be established.

Columns

Name	Type	Constraints
-----	-----	-----
`official_id`	`int8`	Primary Identity
`display_name`	`text`	
`source`	`text`	
`external_ref`	`text`	Nullable

Table `team`

Columns

Name	Type	Constraints
-----	-----	-----
`team_id`	`int8`	Primary Identity
`name`	`text`	Unique

Table `venue`

Columns

Name	Type	Constraints
-----	-----	-----
`venue_id`	`int8`	Primary Identity
`name`	`text`	
`city`	`text`	Nullable

Table `competition`

Columns

Name	Type	Constraints
-----	-----	-----
`competition_id`	`int8`	Primary Identity
`name`	`text`	Unique

Table `app\_user`

Minimal local identity record. Keyed on provider and provider subject rather than on any provider-specific column, so the schema does not depend on the current authentication provider. Personal data remains with that provider.

Columns

```

| Name          | Type          | Constraints |
| ----- | ----- | ----- |
| `app_user_id` | `int8`        | Primary Identity |
| `auth_provider` | `text`        |                |
| `auth_subject` | `text`        |                |
| `display_name` | `text`        | Nullable      |
| `application_role` | `text`        |                |
| `created_at`   | `timestampz`  |                |
| `disabled_at`  | `timestampz`  | Nullable      |

## Table `submission`

### Columns

| Name          | Type          | Constraints |
| ----- | ----- | ----- |
| `submission_id` | `int8`        | Primary Identity |
| `submitted_by` | `int8`        | Nullable      |
| `received_at` | `timestampz`  |                |
| `source_filename` | `text`        | Nullable      |
| `source_sha256` | `bpchar`      | Nullable      |
| `status`       | `submission_status` |                |
| `rejection_detail` | `jsonb`      | Nullable      |

## Table `fixture`

### Columns

| Name          | Type          | Constraints |
| ----- | ----- | ----- |
| `fixture_id` | `int8`        | Primary Identity |
| `source_ref` | `text`        | Unique      |
| `competition_id` | `int8`        | Nullable      |
| `event_match_number` | `int4`        | Nullable      |
| `event_group` | `text`        | Nullable      |
| `event_stage` | `text`        | Nullable      |
| `season`      | `text`        |                |
| `match_type` | `text`        |                |
| `team_type`  | `text`        |                |
| `gender`     | `text`        |                |
| `balls_per_over` | `int2`        |                |
| `scheduled_overs` | `int2`        | Nullable      |
| `venue_id`   | `int8`        | Nullable      |
| `start_date` | `date`        |                |
| `end_date`   | `date`        |                |
| `toss_winner_id` | `int8`        | Nullable      |
| `toss_decision` | `text`        | Nullable      |
| `toss_uncontested` | `bool`        |                |
| `outcome`    | `outcome_kind` |                |
| `winner_id` | `int8`        | Nullable      |
| `eliminator_id` | `int8`        | Nullable      |
| `outcome_by_runs` | `int4`        | Nullable      |
| `outcome_by_wickets` | `int4`        | Nullable      |
| `outcome_method` | `text`        | Nullable      |
| `decided_by_bowl_out` | `bool`        |                |
| `missing_fields` | `_text`        |                |
| `source_version` | `text`        |                |
| `source_revision` | `int4`        |                |
| `first_seen_in` | `int8`        | Nullable      |

## Table `fixture_team`

```

Columns

Name	Type	Constraints
-----	-----	-----
`fixture_id`	`int8`	Primary
`team_id`	`int8`	Primary
`ordinal`	`int2`	

Table `fixture\_squad`

Columns

Name	Type	Constraints
-----	-----	-----
`fixture_id`	`int8`	Primary
`person_id`	`int8`	Primary
`team_id`	`int8`	
`role`	`text`	Nullable

Table `fixture\_official`

Columns

Name	Type	Constraints
-----	-----	-----
`fixture_id`	`int8`	Primary
`official_id`	`int8`	Primary
`role`	`text`	Primary

Table `fixture\_player\_of\_match`

Columns

Name	Type	Constraints
-----	-----	-----
`fixture_id`	`int8`	Primary
`person_id`	`int8`	Primary

Table `innings`

Columns

Name	Type	Constraints
-----	-----	-----
`innings_id`	`int8`	Primary Identity
`fixture_id`	`int8`	
`ordinal`	`int2`	
`batting_team_id`	`int8`	
`is_super_over`	`bool`	
`declared`	`bool`	
`forfeited`	`bool`	
`target_runs`	`int4`	Nullable
`target_overs`	`numeric`	Nullable
`penalty_pre`	`int4`	Nullable
`penalty_post`	`int4`	Nullable

Table `innings\_powerplay`

Columns

Name	Type	Constraints
-----	-----	-----

`innings_id`	`int8`	Primary	
`from_ball`	`numeric`	Primary	
`to_ball`	`numeric`		
`type`	`text`	Primary	

Table `innings\_absent`

Columns

Name	Type	Constraints	
-----	-----	-----	
`innings_id`	`int8`	Primary	
`person_id`	`int8`	Primary	
`reason`	`text`		

Table `innings\_miscounted\_over`

Records overs the umpire miscounted, which legitimately hold five or seven legal balls. The source supplies the ball count as a string in some matches and an integer in others; ingestion coerces it. This table explains an irregularity and is not a derivation input: legal balls are counted from the delivery rows themselves. Some source records also name the umpire responsible. That name is deliberately not stored: it is attribution for an officiating error that nothing in the platform derives from, and officials carry no stable identifier by which it could be resolved reliably.

Columns

Name	Type	Constraints	
-----	-----	-----	
`innings_id`	`int8`	Primary	
`over_number`	`int2`	Primary	
`balls`	`int2`		

Table `delivery`

Columns

Name	Type	Constraints	
-----	-----	-----	
`delivery_id`	`int8`	Primary Identity	
`innings_id`	`int8`		
`over_number`	`int2`		
`position_in_over`	`int2`		
`innings_sequence`	`int4`		
`ball_number`	`text`		
`striker_id`	`int8`		
`non_striker_id`	`int8`		
`bowler_id`	`int8`		
`runs_off_bat`	`int2`		
`runs_extras`	`int2`		
`runs_total`	`int2`		
`non_boundary`	`bool`		
`extra_wides`	`int2`	Nullable	
`extra_noballs`	`int2`	Nullable	
`extra_byes`	`int2`	Nullable	
`extra_legbyes`	`int2`	Nullable	
`extra_penalty`	`int2`	Nullable	
`revision`	`int4`		
`submission_id`	`int8`		
`recorded_at`	`timestampz`		
`superseded_at`	`timestampz`	Nullable	

```
| `superseded_by` | `int8` | Nullable |
```

```
## Table `delivery_wicket`
```

```
### Columns
```

```
| Name          | Type | Constraints |
| ----- | ---- | ----- |
| `wicket_id`   | `int8` | Primary Identity |
| `delivery_id` | `int8` |               |
| `ordinal`     | `int2` |               |
| `kind`        | `text` |               |
| `source_kind` | `text` |               |
| `player_out_id` | `int8` |               |
```

```
## Table `delivery_wicket_fielder`
```

```
### Columns
```

```
| Name          | Type | Constraints |
| ----- | ---- | ----- |
| `wicket_id`   | `int8` | Primary |
| `ordinal`     | `int2` | Primary |
| `person_id`   | `int8` | Nullable |
| `is_substitute` | `bool` |         |
```

```
## Table `delivery_review`
```

```
### Columns
```

```
| Name          | Type | Constraints |
| ----- | ---- | ----- |
| `delivery_id` | `int8` | Primary |
| `by_team_id`  | `int8` |         |
| `umpire_id`   | `int8` | Nullable |
| `batter_id`   | `int8` | Nullable |
| `decision`    | `text` |         |
| `type`        | `text` | Nullable |
```

```
## Table `delivery_replacement`
```

```
### Columns
```

```
| Name          | Type | Constraints |
| ----- | ---- | ----- |
| `replacement_id` | `int8` | Primary Identity |
| `delivery_id`   | `int8` |               |
| `replacement_type` | `text` |               |
| `in_person_id`  | `int8` |               |
| `out_person_id` | `int8` | Nullable |
| `reason`        | `text` | Nullable |
| `role`          | `text` | Nullable |
```

```
## Custom Types / Enums
```

```
### `submission_status`
```

```
`pending` | `accepted` | `rejected`
```

```
### `outcome_kind`
```

```
`won` | `tie` | `draw` | `no result`
```



Thinking

Inspecting newest ZIP and Git issue #58

- Inspected repository ZIP, Git issue, contracts, documentation, and history

Searching the web

github.com

- Inspecting repository context

I'm reviewing the uploaded repository, project brief, AI policy, and backlog to identify the next genuinely incomplete issue. I'll avoid relying on unrelated web results and focus on the project's own documented requirements.

- Inspected repository context

- Searched 1 website

Inspecting Evidence for Issue #58

- Defining shared contracts

I'm mapping the requested API conventions to reusable schemas for identifiers, pagination, filtering, sorting, successful responses, and errors. I'm checking existing database and API patterns first so the contract remains consistent and stable.

- Defined shared contracts

Reviewing Issue #47 API Contracts

- Implemented and validated API contract schemas across shared packages

- Reviewed Sport Analytics API, AI requirements, and Issue B06 evidence

Inspecting event schema conventions and migrations

- Inspected event schemas and AI policy changes, then validated API contracts

✓ Worked for 6m 24s

Yes - **the database schema helps a lot**, and I checked the **newest repo ZIP you just uploaded** before answering.

Your uploaded clone is on clean `main` at:

```
c2f194c
Merge pull request '#89 Add public landing page and application shell' (#90)
9 August 2026
```

And **#58 is definitely not implemented on that** `main`. `packages/contracts/src/` currently only has:

```
health.ts
index.ts
tests/health.test.ts
```

There is no shared identifier/error/pagination/filtering contract yet. So you are safe to take #58.

Even better, I found the **exact original specification for the issue that became #58**. It requires opaque stable identifiers, date/time and event-order conventions, machine-readable errors with field/event details, filtering/sorting conventions, one pagination approach, Zod validation schemas/TypeScript types in `packages/contracts`, tests, and public API examples.

`COMS3011A_Basic_Requirements_Gitea_Issues.md` This directly supports the Sport Analytics brief, which explicitly requires stable identifiers, filtering and pagination. `sport_analytics.pdf`

And yes: **your current DB schema gives us enough information to make the important #58 design decisions properly.**



Do #58 in this exact order

1. Create your branch from the newest `main`.

On your actual local repo:

```
git switch main
git pull origin main
git status
```

Make sure it says your branch is up to date and there are no uncommitted changes.

Then:

```
git switch -c feat/58-api-conventions
```

That branch name is actually the suggested naming pattern from the original issue.

2. Do NOT touch the database for #58.

You do **not** need another migration.

Your current database already tells us how the contract should behave. In particular, your important IDs are PostgreSQL `int8` values:

```
competition_id
fixture_id
team_id
person_id
venue_id
innings_id
delivery_id
submission_id
...
```

But JavaScript cannot safely represent every PostgreSQL `bigint` as a normal number.

Therefore our API decision should be:

```
Database:
fixture_id = bigint

API:
fixtureId = "12345"
```

IDs cross the JSON/API boundary as strings.

They are treated as:

```
stable
immutable
opaque
non-recycled
```

A consumer must **not** use a person's name, team name, competition name, `ball_number`, etc. as an identifier.

Also, `source_ref` remains provenance/source information. It should not replace your own stable API ID.

3. Create this new file:

```
packages/contracts/src/api.ts
```

Put this in it:

```

import { z } from 'zod';

export const DEFAULT_PAGE_LIMIT = 50;
export const MAX_PAGE_LIMIT = 100;

// Database bigint identifiers cross the JSON boundary as strings.
// Consumers must treat the value as opaque and must not derive
// meaning from its representation.
export const apiIdentifierSchema = z.string().min(1);

export const cursorSchema = z.string().min(1);

export const apiDateSchema = z.string().date();

export const apiDateTimeSchema = z.string().datetime();

export const eventSequenceSchema = z.number().int().positive();

export const sortDirectionSchema = z.enum(['asc', 'desc']);

export const paginationQuerySchema = z.object({
  cursor: cursorSchema.optional(),
  limit: z.coerce
    .number()
    .int()
    .min(1)
    .max(MAX_PAGE_LIMIT)
    .default(DEFAULT_PAGE_LIMIT),
});

export const sortingQuerySchema = z.object({
  sort: z.string().min(1).optional(),
  direction: sortDirectionSchema.default('asc'),
});

export const listQuerySchema =
  paginationQuerySchema.merge(sortingQuerySchema);

export const paginationMetadataSchema = z.object({
  nextCursor: cursorSchema.nullable(),
});

export const apiErrorCodeSchema = z
  .string()
  .regex(/^[A-Z][A-Z0-9_]*$/);

export const apiErrorDetailSchema = z.object({
  code: apiErrorCodeSchema,
  message: z.string().min(1),

  // Used for field-level validation failures.
  field: z.string().min(1).optional(),

  // Used when a validation failure belongs to a specific
  // submitted event.
  eventIndex: z.number().int().nonnegative().optional(),
});

export const apiErrorResponseSchema = z.object({
  error: z.object({
    code: apiErrorCodeSchema,
    message: z.string().min(1),
  })
});

```

```

        details: z.array(apiErrorDetailSchema).optional(),
    }},
});

export function createResourceResponseSchema<
    TSchema extends z.ZodTypeAny,
>(schema: TSchema) {
    return z.object({
        data: schema,
    });
}

export function createCollectionResponseSchema<
    TSchema extends z.ZodTypeAny,
>(schema: TSchema) {
    return z.object({
        data: z.array(schema),
        pagination: paginationMetadataSchema,
    });
}

export type ApiIdentifier = z.infer<
    typeof apiIdentifierSchema
>;

export type ApiDate = z.infer<typeof apiDateSchema>;

export type ApiDateTime = z.infer<
    typeof apiDateTimeSchema
>;

export type EventSequence = z.infer<
    typeof eventSequenceSchema
>;

export type SortDirection = z.infer<
    typeof sortDirectionSchema
>;

export type PaginationQuery = z.infer<
    typeof paginationQuerySchema
>;

export type SortingQuery = z.infer<
    typeof sortingQuerySchema
>;

export type ListQuery = z.infer<typeof listQuerySchema>;

export type PaginationMetadata = z.infer<
    typeof paginationMetadataSchema
>;

export type ApiErrorDetail = z.infer<
    typeof apiErrorDetailSchema
>;

export type ApiErrorResponse = z.infer<
    typeof apiErrorResponseSchema
>;

```

I checked this against the current TypeScript/Zod setup in your newest repository. It type-checks and lints with the current configuration.

4. Export the new contracts.

Open:

```
packages/contracts/src/index.ts
```

Currently it says:

```
export * from './health';
```

Change it to:

```
export * from './api';
export * from './health';
```

5. Add contract unit tests.

Create:

```
packages/contracts/src/tests/api.test.ts
```

Use:

```
import { describe, expect, test } from 'vitest';
import { z } from 'zod';

import {
  apiDateSchema,
  apiDateTimeSchema,
  apiErrorResponseSchema,
  apiIdentifierSchema,
  createCollectionResponseSchema,
  createResourceResponseSchema,
  eventSequenceSchema,
  listQuerySchema,
} from '../api';

describe('shared identifier and ordering contracts', () => {
  test('accepts opaque string identifiers', () => {
    expect(
      apiIdentifierSchema.parse('12345678901234567890'),
    ).toBe('12345678901234567890');

    expect(
      apiIdentifierSchema.parse('fixture_01JABC'),
    ).toBe('fixture_01JABC');
  });

  test('rejects an empty identifier', () => {
    expect(
      apiIdentifierSchema.safeParse('').success,
    ).toBe(false);
  });

  test('requires positive event sequence numbers', () => {
    expect(eventSequenceSchema.parse(1)).toBe(1);

    expect(
      eventSequenceSchema.safeParse(0).success,
    ).toBe(false);
  });
});
```

```

    ).toBe(false);
  });
});

describe('shared date and time contracts', () => {
  test('accepts ISO calendar dates and UTC timestamps', () => {
    expect(apiDateSchema.parse('2026-08-09')).toBe(
      '2026-08-09',
    );

    expect(
      apiDateTimeSchema.parse(
        '2026-08-09T12:30:00.000Z',
      ),
    ).toBe('2026-08-09T12:30:00.000Z');
  });

  test('rejects non-ISO date values', () => {
    expect(
      apiDateSchema.safeParse('09/08/2026').success,
    ).toBe(false);
  });
});

describe('listQuerySchema', () => {
  test('applies pagination and sorting defaults', () => {
    expect(listQuerySchema.parse({})).toEqual({
      limit: 50,
      direction: 'asc',
    });
  });

  test('coerces an HTTP limit query parameter', () => {
    expect(
      listQuerySchema.parse({
        limit: '25',
        direction: 'desc',
      }),
    ).toEqual({
      limit: 25,
      direction: 'desc',
    });
  });

  test('rejects an excessive page size', () => {
    expect(
      listQuerySchema.safeParse({
        limit: '101',
      }).success,
    ).toBe(false);
  });
});

describe('shared response schemas', () => {
  const itemSchema = z.object({
    id: apiIdentifierSchema,
    name: z.string().min(1),
  });

  test('validates a resource response', () => {
    const schema =
      createResourceResponseSchema(itemSchema);
  });
});

```

```

expect(
  schema.safeParse({
    data: {
      id: '42',
      name: 'Example',
    },
  }).success,
).toBe(true);
});

test('validates a cursor-paginated collection response', () => {
  const schema =
    createCollectionResponseSchema(itemSchema);

  expect(
    schema.safeParse({
      data: [
        {
          id: '42',
          name: 'Example',
        },
      ],
      pagination: {
        nextCursor: 'opaque-next-cursor',
      },
    }).success,
  ).toBe(true);
});

test('validates field-level and event-level errors', () => {
  expect(
    apiErrorResponseSchema.safeParse({
      error: {
        code: 'VALIDATION_FAILED',
        message: 'The request is invalid.',
        details: [
          {
            code: 'OUT_OF_RANGE',
            field: 'limit',
            message:
              'limit must be between 1 and 100.',
          },
          {
            code: 'INVALID_EVENT',
            eventIndex: 3,
            field: 'runsTotal',
            message:
              'runsTotal is inconsistent with the event.',
          },
        ],
      },
    }).success,
  ).toBe(true);
});

```

This satisfies the issue requirement that the shared contracts have unit tests and validates both field-level and event-level error information. [COMS3011A_Basic_Requirements_Gitea_Issues.md](#)

6. Now document the actual conventions. This is an important part of #58.

Create:

```
docs/api/contracts.md
```

This document should explicitly define these decisions:

Stable IDs

API resource IDs are JSON strings.
They are stable, immutable, opaque and non-recycled.
Mutable names are never IDs.
External `source_ref` values are provenance, not primary API IDs.

Dates

Calendar dates: YYYY-MM-DD

Timestamps

ISO 8601 / RFC 3339 UTC timestamps:
2026-08-09T12:30:00.000Z

Event order

`innings.ordinal` ASC
then
`delivery.innings_sequence` ASC

`ball_number` is display-only and MUST NOT determine event order.

Corrections retain the `innings_sequence` of the event
they supersede.

Pagination

Cursor pagination is the Basic API pagination strategy.

`?limit=50`
`?limit=50&cursor=<opaque-cursor>`

Default limit: 50
Maximum limit: 100

Consumers must treat cursors as opaque.

Collection response

```
{
  "data": [],
  "pagination": {
    "nextCursor": null
  }
}
```

Filtering

Filters use explicit camelCase query parameters.

Examples:

`?competitionId=12`
`?season=2026`
`?startDateFrom=2026-01-01`

Arbitrary SQL-style or database-generated filters are not exposed.

Sorting

?sort=startDate&direction=asc

direction is either:

asc

desc

Every endpoint must whitelist supported sort fields.

A stable identifier must be used as the final tie-breaker
so pagination remains deterministic.

Event endpoints always use event occurrence order.

Errors

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "The request is invalid.",
    "details": [
      {
        "code": "INVALID_EVENT",
        "eventIndex": 3,
        "field": "runsTotal",
        "message": "runsTotal is inconsistent with the event."
      }
    ]
  }
}
```

Also put this at the bottom because we're using AI to write this documentation:

AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].

7. There is one especially important detail from your DB schema to put in the event-order section.

Your migration already says:

```
delivery.innings_sequence
```

is assigned during ingestion, and **a correction inherits the sequence of the delivery it replaces**.

That's excellent for cursor pagination because correcting delivery 35 does not suddenly move it somewhere else in the ordered event stream.

Also:

```
ball_number
```

explicitly cannot be used as the order/identity field because it can repeat.

So #58 should formalise the rule:

Never order public event data by ballNumber alone.

Order:

```
innings ordinal
→
innings sequence
```

8. **Update** `docs/api/overview.md`.

Right now it says:

```
Pagination: cursor pagination for large or changing collections where practical
```

That is actually **too vague for #58**, because its acceptance criteria require you to select **one documented pagination approach**. `COMS3011A_Basic_Requirements_Gitea_Issues.md`

Change it to something like:

- Stable identifiers: opaque, immutable, non-recycled string identifiers
- Pagination: cursor pagination for Basic collection endpoints
- Filtering: explicit documented camelCase query parameters
- Sorting: endpoint-specific whitelisted fields with deterministic tie-breaking
- Errors: consistent machine-readable code, safe message and optional field/event details

And add:

```
See [Shared API Contracts](contracts.md) for the complete identifier,
response, error, filtering, sorting, date/time and pagination conventions.
```

9. **Add the new docs page to** `mkdocs.yml`.

Currently you have:

- API: `api/overview.md`

Replace that with:

- API:
 - Overview: `api/overview.md`
 - Shared Contracts: `api/contracts.md`

10. **Do NOT build competition/fixture/person contracts yet.**

This is an important scope boundary.

Don't start making:

```
competition.ts
fixture.ts
participant.ts
```

as part of #58 unless the actual Gitea issue has subsequently been changed.

#58 establishes the **common rules** they will all use.

Then #47 will later use these building blocks to define things such as:

```
competitionResponseSchema
fixtureResponseSchema
participantResponseSchema
fixtureListQuerySchema
```

This keeps your PR focused.

Also don't do OpenAPI in this issue beyond examples/documentated conventions. **#59 owns the OpenAPI baseline.**

l1. Run your verification before committing.

First format only the files you touched rather than accidentally reformatting the repository:

```
npx prettier --write packages/contracts/src/api.ts packages/contracts/src/tests/api.test.ts
packages/contracts/src/index.ts docs/api/contracts.md docs/api/overview.md mkdocs.yml
```

Then:

```
npm run test:contracts
```

Then:

```
npm run typecheck --workspace=@sport-analytics/contracts
```

Then:

```
npm run lint --workspace=@sport-analytics/contracts
```

Then docs:

```
python -m mkdocs build --strict
```

Then the whole repo:

```
npm run check
```

Finally:

```
git diff --check
git status
git diff
```

Actually read the diff before committing.

l2. Record this AI use correctly, then commit.

Because I've now provided actual code and documentation that you can use, if you use that output this counts as AI-assisted code/document generation. The COMS3011A policy requires the tool/model in the commit for AI-generated code and requires code-generation tools to be declared in the repository README. [ai_policy_20_jul.pdf](#)

Your current README already declares ChatGPT Web, but it lists `GPT-5.6 Thinking`, not this session's model. If you use the code from this answer, update that line to include:

```
ChatGPT-Web[GPT-5.6 Sol]
```

Save/export this chat under your own transcript folder using the current project convention, e.g.:

```
evidence/ai/transcripts/<your-name>/2026-08-09-issue-58-api-conventions.pdf
```

And add a row to:

```
evidence/ai/registers/<your-name>.csv
```

The latest repo is now using the **per-member registers**, rather than having everyone edit the shared CSV.

Then your commit can be:

```
git add packages/contracts/src/api.ts
git add packages/contracts/src/tests/api.test.ts
git add packages/contracts/src/index.ts
```

```
git add docs/api/contracts.md
git add docs/api/overview.md
git add mkdocs.yml
git add README.md
git add evidence/ai
```

Check:

```
git status
```

Then:

```
git commit -m "feat(api): define shared API contracts" -m "Refs #58" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

Push:

```
git push -u origin feat/58-api-conventions
```

When #58 is actually finished

Your acceptance checklist should effectively look like this:

- ✓ IDs have a stable documented format
- ✓ bigint IDs cross API boundary as strings
- ✓ names/source refs are not mutable API IDs
- ✓ ISO date/time rules documented
- ✓ event ordering documented
- ✓ ball_number explicitly not used for ordering
- ✓ reusable error contract
- ✓ field-level errors supported
- ✓ event-level errors supported
- ✓ filtering convention defined
- ✓ sorting convention defined
- ✓ deterministic tie-breaking defined
- ✓ cursor pagination chosen
- ✓ nextCursor response defined
- ✓ Zod schemas added
- ✓ TypeScript types added
- ✓ contract tests added
- ✓ public API docs updated
- ✓ MkDocs nav updated
- ✓ npm run check passes
- ✓ MkDocs strict build passes
- ✓ AI evidence/attribution completed
- ✓ PR reviewed

That is **#58**. You should **not** be writing actual `/competitions` or `/fixtures` endpoints yet. Once this merges, you have laid the exact foundation that your later **#47** endpoints will use. The Sport Analytics brief specifically requires consumers to filter, paginate and hold stable references, so this is useful project work rather than documentation for its own sake.

`sport_analytics.pdf`

If you want to work through it interactively, **start with Steps 1-2 now and show me your terminal after you create `feat/58-api-conventions`**; then I'll take you through the actual files one at a time.

8/9/2026, 2:27:06 PM



confirming none of the steps u did used supabases apis? remember proj says we (including AI) need to hand write our apis

8/9/2026, 2:35:10 PM



Thinking

- **Confirmed no Supabase-generated APIs were used**

✓ Worked for 10s

Yes - **none of the #58 steps I gave you use Supabase-generated APIs.**

They are safe with the project requirement.

The project brief is explicit: you must **design and implement your own API** and may not use something that automatically generates API endpoints, with Supabase specifically given as an example. `project_briefs.pdf` Your project guidance makes the distinction even clearer:

`Frontend -> Your API -> Supabase database = acceptable`

`Frontend -> Automatically generated Supabase API for everything = potentially unacceptable.`

`COMS3011A_Sport_Analytics_Basic_Requirements_Explained.pdf`

For #58, what we are doing is only:

- writing **Zod schemas ourselves**;
- defining TypeScript API types ourselves;
- defining our own error structure;
- deciding our own cursor pagination convention;
- deciding our own sorting/filtering conventions;
- writing our own tests;
- documenting those conventions.

There are **no Supabase API calls at all** in the code I gave you.

For example:

```
export const paginationQuerySchema = z.object({
  cursor: cursorSchema.optional(),
  limit: z.coerce.number()...
});
```

That's just validation code that **we wrote**. Zod isn't generating an API or querying Supabase.

Likewise:

```
export function createCollectionResponseSchema(...)
```

is merely our own definition of what our HTTP responses should look like. It does not create an endpoint.

The rule I will follow with you from now on

For the actual Sport Analytics data API, we should maintain this architecture:

```
Frontend
|
```

```
| HTTP
▼
OUR HAND-WRITTEN API
Express routes
↓
controllers
↓
services
↓
repositories
↓
PostgreSQL / Supabase-hosted DB
```

So when we eventually do #47, **we will manually write** things like:

```
router.get('/competitions', ...)
router.get('/competitions/:competitionId', ...)
router.get('/fixtures', ...)
router.get('/fixtures/:fixtureId', ...)
```

and manually implement:

```
route
→ controller
→ service
→ repository
→ database
```

We will **not** do:

```
Frontend
↓
supabase.from('fixture').select(...)
```

for the public Sport Analytics functionality.

And we will **not** rely on Supabase's automatically generated REST endpoints such as direct PostgREST table APIs.

Important Supabase distinction

Supabase itself is **not banned**.

Your project material specifically says you may still use **Supabase for database storage and authentication**; what you cannot do is let it generate the important Sport Analytics API instead of building that API yourselves.

COMS3011A_Sport_Analytics_Basic_Requirements_Explained.pdf

So these are different:

- ✓ Supabase PostgreSQL hosting
- ✓ Supabase Auth
- ✓ Backend connecting to the database
- ✓ Express API written by your team
- ✓ SQL/repositories written by your team

versus:

- ✗ Supabase auto-generated public CRUD API
- ✗ frontend directly treating Supabase tables as your project API
- ✗ replacing your Express API with PostgREST-generated endpoints

There is actually another reason Supabase Auth is fine: the brief says you **must not write your own authentication system** and should instead rely on an established authentication system/library. [project_briefs.pdf](#)

So for us the guiding rule is:

Supabase can provide infrastructure; it must not provide the Sport Analytics API that you are being assessed on.

And yes - **I will keep that constraint in mind for every API-related step I give you for this project**, especially #58, #47, #67, #52 and the submission endpoints.

8/9/2026, 2:35:22 PM



****Update `docs/api/overview.md`.**** Right now it says: **Pagination:** cursor pagination for large or changing collections where practical That is actually ****too vague for #58****, because its acceptance criteria require you to select ****one documented pagination approach****. **COMS3011A_Basic_Requirements_Gi...** Change it to something like: - Stable identifiers: opaque, immutable, non-recycled string identifiers - **Pagination:** cursor pagination for Basic collection endpoints - **Filtering:** explicit documented camelCase query parameters - **Sorting:** endpoint-specific whitelisted fields with deterministic tie-breaking - **Errors:** consistent machine-readable code, safe message and optional field/event details And add: **Markdown** See [Shared API Contracts] (contracts.md) for the complete identifier, response, error, filtering, sorting, date/time and pagination conventions.

i dont understand step 8, tell me exactly what to do?

8/9/2026, 2:39:42 PM



Thinking

Inspecting API Overview Documentation

- Inspected and read API overview documentation

✓ Worked for 13s

Yep — step 8 is just editing **one existing documentation file**. You are not writing any backend code here.

Open:

```
docs/api/overview.md
```

In your newest repo, the top part currently looks like this:

```
# API overview
```

```
## Product boundary
```

The API is a primary product. It must be designed and implemented by the team as HTTP endpoints. Generated database endpoints must not be used as the application API. Supabase may provide managed authentication, but application data must pass through the handwritten backend.

```
## Initial conventions
```

- Base path: `/api/v1`
- Format: JSON unless returning a documented dataset file
- Stable identifiers: opaque, non-recycled IDs
- Pagination: cursor pagination for large or changing collections where practical
- Filtering: explicit documented query parameters
- Errors: consistent machine-readable code, safe message, and optional field details
- Authentication: established provider/library for users; separate API-consumer credentials when introduced
- Versioning: URL major version initially, with a documented deprecation path before any retirement

You only need to replace the `## Initial conventions` **section**.

So delete this:

```
## Initial conventions
```

- Base path: `/api/v1`
- Format: JSON unless returning a documented dataset file
- Stable identifiers: opaque, non-recycled IDs
- Pagination: cursor pagination for large or changing collections where practical
- Filtering: explicit documented query parameters
- Errors: consistent machine-readable code, safe message, and optional field details

- Authentication: established provider/library for users; separate API-consumer credentials when introduced
- Versioning: URL major version initially, with a documented deprecation path before any retirement

and replace it with this exact version:

Initial conventions

- Base path: `/api/v1`
- Format: JSON unless returning a documented dataset file
- Stable identifiers: opaque, immutable, non-recycled string identifiers
- Pagination: cursor pagination for Basic collection endpoints
- Filtering: explicit documented camelCase query parameters
- Sorting: endpoint-specific whitelisted fields with deterministic tie-breaking
- Errors: consistent machine-readable code, safe message, and optional field or event details
- Authentication: established provider/library for users; separate API-consumer credentials when introduced
- Versioning: URL major version initially, with a documented deprecation path before any retirement

See [Shared API Contracts](contracts.md) for the complete identifier, response, error, filtering, sorting, date/time, event-ordering, and pagination conventions.

So the beginning of the file should now look exactly like:

API overview

Product boundary

The API is a primary product. It must be designed and implemented by the team as HTTP endpoints. Generated database endpoints must not be used as the application API. Supabase may provide managed authentication, but application data must pass through the handwritten backend.

Initial conventions

- Base path: `/api/v1`
- Format: JSON unless returning a documented dataset file
- Stable identifiers: opaque, immutable, non-recycled string identifiers
- Pagination: cursor pagination for Basic collection endpoints
- Filtering: explicit documented camelCase query parameters
- Sorting: endpoint-specific whitelisted fields with deterministic tie-breaking
- Errors: consistent machine-readable code, safe message, and optional field or event details
- Authentication: established provider/library for users; separate API-consumer credentials when introduced
- Versioning: URL major version initially, with a documented deprecation path before any retirement

See [Shared API Contracts](contracts.md) for the complete identifier, response, error, filtering, sorting, date/time, event-ordering, and pagination conventions.

Current endpoints

Then **leave everything from** `## Current endpoints` **downward unchanged.**

The reason for the changes is simple:

- "where practical" was too vague — #58 should choose one pagination approach.
- We are explicitly choosing **cursor pagination**.
- We are stating API IDs are **strings**.
- We are adding the missing **sorting convention**.
- We are acknowledging that errors can point to a **field or a particular submitted event**.
- The final link sends readers to the detailed `docs/api/contracts.md` file you created in step 6.

This does **not** use Supabase APIs. In fact, your existing first paragraph explicitly preserves the required architecture:

```
application data must pass through the handwritten backend
```

After saving the file, you can check what changed with:

```
git diff docs/api/overview.md
```

If you paste that output here, I can check it before you move to step 9.

8/9/2026, 2:39:43 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> python -m mkdocs build --strict
C:\Python314\python.exe: No module named mkdocs
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> python -m mkdocs.yml build --strict
C:\Python314\python.exe: Error while finding module specification for 'mkdocs.yml' (ModuleNotFoundError: No module named 'mkdocs')
```

8/9/2026, 2:43:11 PM



Thinking



Inspecting project dependencies for MkDocs setup

- Inspected MkDocs dependencies and requirements documentation

✓ Worked for 17s

That error just means **MkDocs is not installed in your current Python environment**. Your repo already includes the correct dependency file, so **don't install random packages manually**.

From the repo root, run:

```
python -m pip install -r requirements-docs.txt
```

That should install the version your project expects:

```
mkdocs-material>=9,<10
```

Then verify it installed:

```
python -m mkdocs --version
```

If you see a version number, run your original command again:

```
python -m mkdocs build --strict
```

Important

This command was wrong:

```
python -m mkdocs.yml build --strict
```

`mkdocs.yml` is just the configuration file. You don't execute it as a Python module.

So the sequence you want is:

```
python -m pip install -r requirements-docs.txt
python -m mkdocs --version
python -m mkdocs build --strict
```

Your repo's own README actually specifies exactly these commands for building the documentation, so you're following the project's intended setup.

If the `pip install` command throws an error, paste that output here and we'll fix that next.

8/9/2026, 2:43:12 PM





```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool\@0.1.0 check
> npm run structure\check && npm run format\check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool\@0.1.0 structure\check
> node scripts/check-required-files.mjs

Repository structure check passed (20 required files).

> sport-analytics-tool\@0.1.0 format\check
> prettier --check .

Checking formatting...
[warn] apps/backend/package.json
[warn] apps/backend/scripts/ingest-match.ts
[warn] apps/backend/scripts/sql.ts
[warn] apps/backend/scripts/validate-match.ts
[warn] apps/backend/tests/database/transaction.database.test.ts
[warn] docs/deployment/cloudflare\_pages.md
[warn] docs/development/testing.md
[warn] evidence/ai/registers/README.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-04-issue-38-doc-structure-for-
files.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-05-general-merge-issue-
resolution.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-05-general-project-name-
suggestions.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-05-issue-1517-azure-
deployment-strategy.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-05-issue-30-project-file-
structure-documentation.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-general-event-data-
submission-process.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-general-event-model-design-
review\.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-general-pr-merge-
instructions.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-general-sport-selection-for-
analytics.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-issue-30-ai-transcript-
structure.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-06-issue-72-testing-
infrastructure-setup.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-07-general-git-merge-conflict-
resolution.md
[warn] evidence/ai/transcripts/dean-feldman/2026-08-07-issue-72-testing-foundation-
setup.md
[warn] evidence/ai/transcripts/gabriel-raz/AI transcript Issue #14.md
[warn] evidence/decisions/ADR-003-database-host-connection-and-migrations.md
[warn] evidence/validation/729307-published-figures.md
[warn] Code style issues found in 24 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/9/2026, 2:46:08 PM



Thinking



Inspecting repository formatting issues and next steps

- **Validating targeted files**

I'm noting that formatting warnings appear confined to existing files, while the modified contract files pass. I'll avoid rewriting unrelated files and recommend targeted formatting, linting, tests, type checks, and build validation.

- **Validated targeted files**

- **Inspected repository formatting configuration and checked Prettier on main**

✓ Worked for 30s

This is actually **good news for your #58 work**.

`npm run check` is failing at the **repository-wide Prettier check**, but **none of the #58 files we just created/edited appear in the warning list**.

I also checked the newest ZIP you uploaded: it is a clean `main` at commit `c2f194c`. The files Prettier is complaining about are existing repository files, not the #58 files we worked on.

Do NOT run this

Do **not** run:

```
npm.cmd run format
```

or:

```
npx.cmd prettier --write .
```

That would modify all 24 unrelated files and pollute your #58 PR with other people's formatting changes.

Do this now

First, confirm exactly what **you** changed:

```
git status --short
```

Then:

```
git diff --name-only
```

Your #58 files should be things like:

```
packages/contracts/src/api.ts
packages/contracts/src/tests/api.test.ts
packages/contracts/src/index.ts
docs/api/contracts.md
docs/api/overview.md
mkdocs.yml
```

plus README/AI evidence if you've already reached those steps.

1. Check only your #58 formatting

Run:

```
npx.cmd prettier --check packages/contracts/src/api.ts packages/contracts/src/tests/api.test.ts
packages/contracts/src/index.ts docs/api/contracts.md docs/api/overview.md mkdocs.yml
```

You want:



All matched files use Prettier code style!

If that passes, **your #58 files are formatted correctly.**

2. Run the contracts tests

```
npm.cmd run test:contracts
```

This is particularly important for #58.

3. Type-check the contracts package

```
npm.cmd run typecheck --workspace=@sport-analytics/contracts
```

4. Lint the contracts package

```
npm.cmd run lint --workspace=@sport-analytics/contracts
```

5. Build the contracts package

```
npm.cmd run build --workspace=@sport-analytics/contracts
```

6. Check the documentation

Now that you've installed MkDocs:

```
python -m mkdocs build --strict
```

That needs to pass because you edited the API documentation and `mkdocs.yml`.

7. Continue the rest of the repo checks manually

Because the global formatting stage is currently blocking `npm run check`, run the remaining stages yourself:

```
npm.cmd run lint
npm.cmd run typecheck
npm.cmd run test
npm.cmd run build
```

You already know:

```
structure:check 
```

So effectively we're checking:

structure	
your formatting	← check separately
lint	← run
typecheck	← run
tests	← run
build	← run
MkDocs	← run

Why `npm run check` stopped

Your script is:

```
structure:check
  &&
format:check
  &&
lint
  &&
typecheck
  &&
test
  &&
build
```

The `&&` means:

only run the next command if the previous one succeeds.

So because:

```
format:check ❌
```

everything after it **never ran**.

It doesn't currently mean:

```
lint ❌
typecheck ❌
tests ❌
build ❌
```

They simply weren't attempted.

For your PR

If all your targeted checks pass but the full command still fails only because of those existing formatting files, **do not fix all 24 as part of #58**.

We can mention in the PR verification section something like:

```
### Verification

- `npm run structure:check` ✅
- #58 files pass Prettier ✅
- contracts lint ✅
- contracts typecheck ✅
- contract tests ✅
- contracts build ✅
- `python -m mkdocs build --strict` ✅
- repository lint/typecheck/test/build ✅
- `npm run check` is currently blocked at the repository-wide
  Prettier check by pre-existing formatting violations on `main`
  outside the scope of #58.
```

But **don't write that yet**. Let's actually run the checks first.

Start with:

```
npx.cmd prettier --check packages/contracts/src/api.ts packages/contracts/src/tests/api.test.ts
packages/contracts/src/index.ts docs/api/contracts.md docs/api/overview.md mkdocs.yml
```

Send me what that gives you.

8/9/2026, 2:46:09 PM



```
S C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run test
```

```
> sport-analytics-tool@0.1.0 test
> npm run test:unit && npm run test:frontend && npm run test:api && npm run
test:contracts
```

```
> sport-analytics-tool@0.1.0 test:unit
> npm run test:unit --workspace=@sport-analytics/backend
```

```
> @sport-analytics/backend@0.1.0 test:unit
> vitest run tests/unit
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend
```

```
✓ tests/unit/domain-builders.test.ts (1)
✓ tests/unit/test-database-safety.test.ts (6)
```

```
Test Files 2 passed (2)
```

```
Tests 7 passed (7)
```

```
Start at 14:48:06
```

```
Duration 933ms (transform 959ms, setup 0ms, collect 967ms, tests 5ms,
environment 0ms, prepare 293ms)
```

```
> sport-analytics-tool@0.1.0 test:frontend
> npm run test --workspace=@sport-analytics/frontend
```

```
> @sport-analytics/frontend@0.1.0 test
> vitest run
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend
```

```
✓ src/App.test.tsx (5)
✓ public landing page (5)
✓ renders the branded public page without requesting API data
✓ uses a stored theme before the operating-system preference
✓ uses the operating-system preference when no theme is stored
✓ falls back to Day Match when no stored or system preference is available
✓ persists a manual theme selection
```

```
Test Files 1 passed (1)
```

```
Tests 5 passed (5)
```

```
Start at 14:48:08
```

```
Duration 3.02s (transform 90ms, setup 309ms, collect 283ms, tests 93ms,
environment 1.92s, prepare 115ms)
```

```
> sport-analytics-tool@0.1.0 test:api
> npm run test:api --workspace=@sport-analytics/backend
```

```
> @sport-analytics/backend@0.1.0 test:api
> vitest run tests/api tests/auth.test.ts tests/health.test.ts
```

```
RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend
```

```
> tests/auth.test.ts (0)
> tests/health.test.ts (0)
```

Failed Suites 2

```
FAIL tests/auth.test.ts [ tests/auth.test.ts ]
```

```
FAIL tests/health.test.ts [ tests/health.test.ts ]
Error: Failed to load url @supabase/supabase-js (resolved id: @supabase/supabase-js)
in C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend/src/auth/supabase-auth.ts. Does the file exist?
> loadAndTransform ../../node_modules/vite/dist/node/chunks/dep-
BK3b2jBa.js:51969:17
```

Test Files 2 failed (2)

Tests no tests

Start at 14:48:12

Duration 1.49s (transform 563ms, setup 0ms, collect 0ms, tests 0ms, environment 1ms, prepare 268ms)

npm error Lifecycle script `test:api` failed with error:

npm error code 1

npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend

npm error workspace @sport-analytics/backend@0.1.0

npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend

npm error command failed

npm error command C:\Windows\system32\cmd.exe /d /s /c vitest run tests/api tests/auth.test.ts tests/health.test.ts

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run build

> sport-analytics-tool@0.1.0 build

> npm run build --workspace=@sport-analytics/contracts && npm run build --workspace=@sport-analytics/backend && npm run build --workspace=@sport-analytics/frontend

> @sport-analytics/contracts@0.1.0 build

> tsc -p tsconfig.json

> @sport-analytics/backend@0.1.0 build

> tsc -p tsconfig.json

src/auth/supabase-auth.ts:1:30 - error TS2307: Cannot find module '@supabase/supabase-js' or its corresponding type declarations.

1 import { createClient } from '@supabase/supabase-js';

~~~~~

Found 1 error in src/auth/supabase-auth.ts:1

npm error Lifecycle script `build` failed with error:

npm error code 2

npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend

npm error workspace @sport-analytics/backend@0.1.0

npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\backend

npm error command failed

npm error command C:\Windows\system32\cmd.exe /d /s /c tsc -p

8/9/2026, 2:48:41 PM



## Thinking



Inspecting Package Dependencies and Lockfiles



- **Fixing the dependency**

I'm treating this as a stale local install, not a code defect or issue #58. From the project root, I'd run `npm ci` for a clean lockfile-based install, then rerun the contract test.

- **Fixed the dependency**

-  **Checking Supabase Dependency Inclusions**

- **Inspected package dependencies, lockfiles, and Supabase inclusions**

✓ Worked for 30s

This is **not a #58 code problem**. Your local install is missing the Supabase package that the backend already declares as a dependency.

I checked the newest repo ZIP: `apps/backend/package.json` already contains:

```
"@supabase/supabase-js": "^2.112.1"
```

and the root `package-lock.json` also contains it. So **do not edit** `package.json` **and do not add Supabase manually**.

From the repo root, do this:

```
npm.cmd install
```

Then verify the dependency is actually installed:

```
npm.cmd ls @supabase/supabase-js --workspace=@sport-analytics/backend
```

You should see something showing `@supabase/supabase-js@2.112.1` or similar.

Then rerun just the failed API tests:

```
npm.cmd run test:api
```

Then test the backend build:

```
npm.cmd run build --workspace=@sport-analytics/backend
```

After that, specifically verify your #58 work:

```
npm.cmd run test:contracts
npm.cmd run typecheck --workspace=@sport-analytics/contracts
npm.cmd run lint --workspace=@sport-analytics/contracts
npm.cmd run build --workspace=@sport-analytics/contracts
```

If `npm.cmd install` still leaves the package missing, use the clean-install fallback:

```
npm.cmd ci
```

`npm ci` will rebuild `node_modules` from the existing `package-lock.json`, so it is a good fix for a stale local dependency folder.

Also, this Supabase dependency is being used for **Supabase Auth**, not for generating your Sport Analytics data API. That is compatible with the project brief, which says authentication should use an established system while your application API must be hand-written. `project_briefs.pdf`

So right now:

|                              |       |
|------------------------------|-------|
| #58 code issue?              | ✗ No  |
| Missing local dependency?    | ✓ Yes |
| Need to change package.json? | ✗ No  |
| Need npm install / npm ci?   | ✓ Yes |

Run:

```
npm.cmd install
```

first and send me the output if anything errors.

*8/9/2026, 2:48:43 PM*



ok, so now im ready for updating the ai usage studd, and then commit? please give me step by step

8/9/2026, 2:51:13 PM



### Thinking

#### Inspecting latest repository AI conventions

- **Updating project records**

I'm identifying Shayna's project row and checking the README and API documentation for the required naming. I'll preserve existing model references, add the current assistant attribution, and explain how to save this conversation as evidence.

- **Updated project records**

#### Inspecting API Docs And Transcript Guidelines

- **Planning verification steps**

I'm preparing a step-by-step sequence: confirm focused checks pass, update the README and relevant API document declarations, add the required attribution, review the diff, and commit only after verification succeeds.

- **Planned verification steps**

#### Checking documentation build and tracked site files

- **Inspected repository AI conventions, evidence documentation, and deployment history**

✓ Worked for 1m 26s

Yes. Assuming the dependency problem is fixed and your #58 checks now pass, the remaining flow is:

**AI evidence → final diff check → stage only #58 files → commit → push.**

There is one small AI-policy correction to what I told you earlier: because we also edited `docs/api/overview.md` with AI, **that document needs an AI declaration too.**

## 1. First confirm your work is actually passing

Before doing the AI evidence, run these:

```
npm.cmd run test:contracts
npm.cmd run typecheck --workspace=@sport-analytics/contracts
npm.cmd run lint --workspace=@sport-analytics/contracts
npm.cmd run build --workspace=@sport-analytics/contracts
python -m mkdocs build --strict
```

And, now that you've fixed the missing package:

```
npm.cmd run test:api
npm.cmd run build
```

If those pass, continue.

You already know the full:

```
npm.cmd run check
```

may still stop on **pre-existing Prettier problems outside #58**. Don't fix those unrelated files in this PR.

## 2. Remove generated `site/` changes



Because you ran:

```
python -m mkdocs build --strict
```

MkDocs may have modified the generated `site/` directory.

We are using the build to **verify** the documentation, not to include generated website files in #58.

Run:

```
git status --short
```

If you see files beginning with:

```
M site/...
```

or new files under `site/`, run:

```
git restore site
```

Then:

```
git status --short
```

Do **not** accidentally commit generated `site/` files.

---

### 3. Add the AI declaration to `docs/api/contracts.md`

Open:

```
docs/api/contracts.md
```

At the very bottom, make sure this exists:

```
## AI Declaration
```

```
The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].
```

If it's already there, leave it.

---

### 4. Add the AI declaration to `docs/api/overview.md`

We also changed this document using this ChatGPT session.

At the **very bottom** of:

```
docs/api/overview.md
```

add:

```
## AI Declaration
```

```
The preceding document was reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Sol].
```

You do **not** need AI comments inside the TypeScript files. AI-generated **code is attributed through the commit**.

---

## 5. Update the repository README's AI usage

Open:

```
README.md
```

Find:

```
## AI usage
```

Your newest `main` currently says:

```
This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking] and Codex[GPT-5].
```

Because you have now used **ChatGPT Web with GPT-5.6 Sol to generate #58 code**, it must also be listed.

Change that line to:

```
This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking], ChatGPT-Web[GPT-5.6 Sol], and Codex[GPT-5].
```

Leave these lines unchanged:

```
This repository does not currently use AI in-line editing tools.
```

```
This repository does not currently use AI code review.
```

That's important: **we did not use an AI code-review tool** here. We used ChatGPT for planning, code generation, documentation and troubleshooting.

## 6. Save this ChatGPT transcript

Your folder already exists:

```
evidence/ai/transcripts/shayna-unterslak/
```

Because this conversation has included both planning around #47 **and** the actual #58 work, there are two acceptable options.

If you can save/export only the #58 portion, name it:

```
2026-08-09-issue-58-api-conventions.pdf
```

and put it here:

```
evidence/ai/transcripts/shayna-unterslak/2026-08-09-issue-58-api-conventions.pdf
```

If your browser export includes this **whole longer project conversation**, use the repository's continuous-session convention instead:

```
evidence/ai/transcripts/shayna-unterslak/2026-08-09-continuous-project-session.pdf
```

That's more accurate than pretending the entire transcript is only about #58.

### Before adding the PDF

Open it and quickly check that it contains **no**:

- passwords;
- Supabase keys;
- database connection strings;
- API tokens;
- private keys.

The DB schema you pasted is fine. **Credentials would not be.**

---

## 7. Add your row to your personal AI register

Do **not** edit the old shared CSV.

Open:

```
evidence/ai/registers/shayna-underslak.csv
```

It currently only has the header, so you will add your first data row beneath it.

### If you used the issue-specific transcript filename

Paste this as **one single CSV row**:

```
"2026-08-09","Shayna Underslak","ChatGPT Web","GPT-5.6 Sol","Planning; code generation; documentation; technical guidance; debugging","Issue #58: define shared API contracts and conventions","Shared Zod and TypeScript API contracts; identifier, pagination, sorting, filtering, date/time, event-ordering and error conventions; contract tests; API documentation updates","Reviewed the generated contracts against the current database schema and handwritten API requirement; ran contract tests, lint, type-check and build; built MkDocs in strict mode; verified backend API tests and project build after restoring the missing installed dependency; repository-wide check remains separately affected by pre-existing unrelated formatting violations on main","evidence/ai/transcripts/shayna-underslak/2026-08-09-issue-58-api-conventions.pdf; Issue #58; Pull Request and commit references pending"
```

### If you used the continuous-session filename

Use exactly the same row but change the final field to:

```
"evidence/ai/transcripts/shayna-underslak/2026-08-09-continuous-project-session.pdf; Issue #58; Pull Request and commit references pending"
```

**Important:** only use the verification wording above if those checks actually passed.

If `test:api` or `build` still fails, tell me and I'll adjust the row so you don't falsely claim verification.

---

## 8. Format the files you changed

Run:

```
npx.cmd prettier --write packages/contracts/src/api.ts packages/contracts/src/tests/api.test.ts packages/contracts/src/index.ts docs/api/contracts.md docs/api/overview.md mkdocs.yml README.md evidence/ai/registers/shayna-underslak.csv
```

You don't need to run Prettier on the PDF.

Then verify only your files:

```
npx.cmd prettier --check packages/contracts/src/api.ts packages/contracts/src/tests/api.test.ts packages/contracts/src/index.ts docs/api/contracts.md docs/api/overview.md mkdocs.yml README.md evidence/ai/registers/shayna-underslak.csv
```

You want:

```
All matched files use Prettier code style!
```

---

## 9. Run one final verification after the AI/doc edits

Because you just changed documentation and the README:

```
npm.cmd run test:contracts
npm.cmd run typecheck --workspace=@sport-analytics/contracts
npm.cmd run lint --workspace=@sport-analytics/contracts
```

```
npm.cmd run build --workspace=@sport-analytics/contracts
python -m mkdocs build --strict
```

Again, the MkDocs build may alter `site/`.

Afterwards:

```
git restore site
```

Then:

```
git diff --check
```

No output from `git diff --check` = good.

---

## 10. Look at exactly what you're about to commit

Run:

```
git status --short
```

I would expect your #58 changes to include approximately:

```
M README.md
M docs/api/overview.md
M mkdocs.yml
M packages/contracts/src/index.ts
M evidence/ai/registers/shayna-underslak.csv
?? docs/api/contracts.md
?? packages/contracts/src/api.ts
?? packages/contracts/src/tests/api.test.ts
?? evidence/ai/transcripts/shayna-underslak/2026-08-09-...
```

Then:

```
git diff
```

Read it.

Make sure you don't see unrelated people's files being modified.

---

## 11. Stage only your #58 files

Don't use `git add .`

Run:

```
git add packages/contracts/src/api.ts
git add packages/contracts/src/tests/api.test.ts
git add packages/contracts/src/index.ts
git add docs/api/contracts.md
git add docs/api/overview.md
git add mkdocs.yml
git add README.md
git add evidence/ai/registers/shayna-underslak.csv
```

Then add whichever transcript filename you actually used.

For the issue-specific one:

```
git add "evidence/ai/transcripts/shayna-underslak/2026-08-09-issue-58-api-conventions.pdf"
```

Or the continuous one:

```
git add "evidence/ai/transcripts/shayna-unterslak/2026-08-09-continuous-project-session.pdf"
```

## 12. Check staged files BEFORE committing

Run:

```
git status
```

Then:

```
git diff --cached --stat
```

And ideally:

```
git diff --cached
```

The PDF won't display as text, which is normal.

Make sure there is:

**no** `site/`, no unrelated backend work, no unrelated formatting changes.

## 13. Commit with the required AI attribution

Use:

```
git commit -m "feat(api): define shared API contracts" -m "Refs #58" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Sol]"
```

That is important because the course AI policy requires AI-generated code to be attributed in the commit.

Then confirm:

```
git log -1 --format=full
```

You should see something like:

```
feat(api): define shared API contracts
```

```
Refs #58
```

```
Assisted-by: ChatGPT-Web[GPT-5.6 Sol]
```

## 14. Push your branch

Assuming your branch is:

```
feat/58-api-conventions
```

run:

```
git push -u origin feat/58-api-conventions
```

## 15. One thing remains after the commit

Your AI register currently says:

```
Pull Request and commit references pending
```

That's okay **for this first commit**, because the commit hash did not exist yet.

Once you've pushed and created the PR, you should update that register entry with:

```
Issue #58; PR #<number>; commit <short-hash>
```

That can go in a **small evidence follow-up commit on the same branch** before the PR is merged.

So don't merge the PR with "pending" still sitting there.

### Your immediate sequence

1. Add AI declarations
2. Update README model
3. Save transcript
4. Add Shayna CSV row
5. Prettier targeted files
6. Re-run #58 checks
7. git restore site
8. git diff --check
9. git status
10. git diff
11. git add ONLY #58 files
12. git diff --cached
13. commit with Assisted-by
14. push
15. create PR
16. replace pending AI references before merge

**Do Steps 1-7 first. Then send me your** `git status --short` **output before you stage anything**, and I'll check that you're not about to commit any accidental files.

8/9/2026, 2:51:14 PM

